

PyQUDA

QUDA 的 Python/Cython 接口使用手册

张鑫

2026 年 7 月 20 日

摘要

PyQUDA 是 QUDA (用于格点 QCD 的 GPU 加速库) 的 Python/Cython 封装。它结合了 Python 的易用性和 QUDA 的 GPU 高性能计算能力, 支持多 GPU 分布式计算 (通过 MPI for Python), 并提供完整的格点 QCD 计算功能: 夸克传播子求解 (Wilson/Clover/HISQ 费米子作用量, 含多重网格支持)、混合蒙特卡洛 (HMC)、规范固定、规范涂抹、费米子涂抹、梯度流以及 10 余种格点数据格式的文件 I/O。本手册详细介绍 PyQUDA 的安装配置、三层架构、核心功能、示例演示以及完整的 API 接口清单。

目录

1	项目概述	2
1.1	主要特性	2
1.2	三层架构	2
1.3	PyPI 发布	3
2	安装	3
2.1	前置要求	3
2.2	从 PyPI 安装 (推荐)	4
2.3	从源码安装	4
2.4	开发模式安装 (原地构建)	4
3	初始化与网格拓扑	5
3.1	基本初始化	5
3.2	LatticeInfo	5
3.3	进程网格映射类型	6

目录	2
4 格点场类型系统	6
4.1 场类型层次	6
4.2 场的基本操作	7
4.3 场的类型 (pyquda.field 扩展)	7
5 Dirac 算子	8
5.1 算子层次结构	8
5.2 创建和使用 Dirac 算子	8
5.3 传播子求解	9
5.4 Dirac 算子方法	10
5.5 精度控制	10
6 HMC (混合蒙特卡洛)	10
6.1 HMC 使用示例	10
6.2 内置积分器	12
6.3 HMC 类方法	12
6.4 费米子作用量	13
7 规范操作	13
7.1 规范涂抹	13
7.2 梯度流	14
7.3 规范固定	14
7.4 物理观测量	14
8 文件 I/O	15
8.1 支持的格式	15
8.2 加载/保存规范组态	15
8.3 校验和	16
9 Gamma 矩阵系统	16
9.1 $\text{Gamma} \times$ 传播子	17
10 源构造	17
10.1 高斯/Wuppertal 涂抹	18
11 插件系统	18
11.1 内置插件	18
11.2 pycontract — 夸克收缩	18
11.3 pygwu — Overlap 费米子	19

目录	3
12 示例演示	19
12.1 示例 1: 淬火 HMC	19
12.2 示例 2: 夸克传播子与 介子两点函数	20
12.3 示例 3: 介子和质子两点函数	20
12.4 示例 4: PCAC 验证	20
12.5 示例 5: 介子色散关系	20
13 QUDA 底层 API	20
13.1 参数结构体	20
13.2 主要 QUDA 函数	21
14 高级功能	22
14.1 动量相位	22
14.2 Wilson 圈计算	22
14.3 规范场转换	23
14.4 FFT 工具	23
15 API 接口清单	23
15.1 初始化模块 (pyquda/__init__.py)	23
15.2 场模块 (pyquda_comm/field.py)	24
15.3 Dirac 算子模块 (pyquda/dirac/)	24
15.4 作用量模块 (pyquda/action/)	25
15.5 HMC 模块 (pyquda/hmc.py)	26
15.6 Gamma 矩阵模块 (pyquda/gamma.py)	27
15.7 源构造模块 (pyquda_utils/source.py)	27
15.8 I/O 模块 (pyquda_io/)	28
15.9 工具模块 (pyquda_utils/core.py)	28
15.10 插件模块 (pyquda_plugins/)	29
16 性能基准测试	29
17 维护说明	29
17.1 更新 QUDA API	30

1 项目概述

PyQUDA 是一个用 Cython 编写的 QUDA Python 封装库，由 CLQCD 合作组开发和维护。该项目旨在利用 Python 中的优化线性代数库（如 CuPy、PyTorch、DPNP）与 QUDA 的 GPU 计算能力相结合，实现高性能的格点 QCD 研究运算。

1.1 主要特性

- **多 GPU 支持**：通过 mpi4py 实现 MPI 分布式计算，支持 4D 进程网格拓扑
- **多后端 GPU 加速**：支持 CuPy (NVIDIA CUDA/AMD HIP)、PyTorch (CUDA/HIP/SYCL)、DPNP (Intel GPU) 及 numpy (CPU 回退)
- **丰富的文件 I/O**：支持 Chroma、MILC、KYU、XQCD、NERSC、OpenQCD、ILDG、NPY、HDF5 等 10 余种格式
- **多种夸克作用量**：Wilson、Clover (各向同性/各向异性)、HISQ 费米子
- **多重网格求解器**：支持 Wilson 和 Clover 费米子的多重网格加速
- **HMC 模拟**：纯规范场、1/2 味 Clover 费米子、N 味 HISQ 费米子
- **规范操作**：规范固定、规范涂抹 (APE/Stout/HYP)、梯度流 (Wilson/Symanzik)
- **费米子涂抹**：高斯/Wuppertal 涂抹

1.2 三层架构

PyQUDA 采用清晰的三层架构设计：

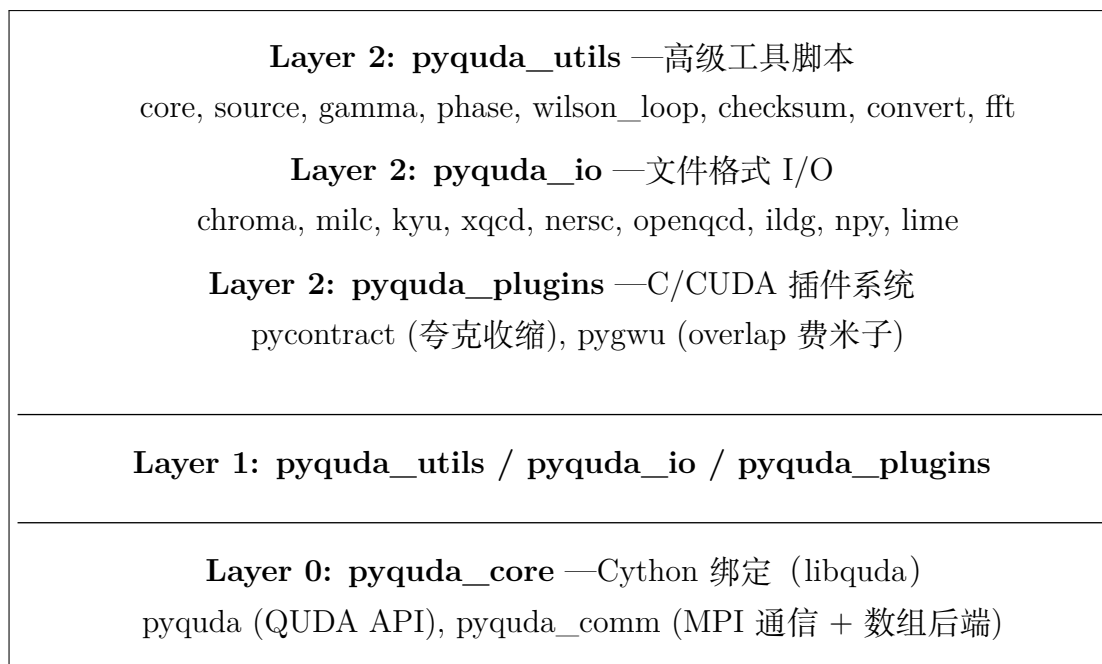


图 1: PyQUDA 三层架构

1.3 PyPI 发布

两个独立的 PyPI 包:

- **PyQUDA** (核心层): 依赖 mpi4py、numpy, 提供 pyquda 和 pyquda_comm 模块
- **PyQUDA-Utills**(工具层): 依赖 PyQUDA, 提供 pyquda_utils、pyquda_io、pyquda_plugins 模块

2 安装

2.1 前置要求

表 1: 安装前置要求

组件	说明
QUDA	编译安装的 QUDA 库 (develop 分支, \$QUDA_PATH 指向 build/usqcd)
CUDA/ROCm	GPU 计算工具链
MPI	支持 GPU 直连的 MPI 实现 (如 OpenMPI)
Python	≥ 3.8
pip 依赖	numpy, mpi4py, cupy (或 torch/dpnp)
编译依赖	Cython, setuptools, wheel

2.2 从 PyPI 安装 (推荐)

```
1 # 确保 QUDA_PATH 指向 QUDA 安装目录
2 export QUDA_PATH=/path/to/quda/build/usqcd
3
4 # 安装核心包和工具包
5 python3 -m pip install pyquda pyquda-utils
```

Listing 1: 从 PyPI 安装

2.3 从源码安装

```
1 export QUDA_PATH=/path/to/quda/build/usqcd
2
3 # 克隆仓库 (含子模块)
4 git clone --recursive https://github.com/CLQCD/PyQUDA.git
5 cd PyQUDA
6
7 # 安装核心 Cython 绑定
8 cd pyquda_core
9 python3 -m pip install .
10 cd ..
11
12 # 安装工具层
13 python3 -m pip install .
```

Listing 2: 从源码编译安装

2.4 开发模式安装 (原地构建)

```
1 export QUDA_PATH=/path/to/quda/build/usqcd
2 git clone --recursive https://github.com/CLQCD/PyQUDA.git
3 cd PyQUDA
4
5 # 原地构建 Cython 扩展
6 cd pyquda_core
7 python3 setup.py build_ext --inplace
8 cd ..
9
10 # 创建符号链接
```

```

11 python3 setup.py egg_info
12 ln -s pyquda_core/pyquda_comm ./
13 ln -s pyquda_core/pyquda ./

```

Listing 3: 原地构建（开发模式）

注意：从其他目录运行脚本时需将 PyQUDA 根目录添加到 `sys.path`。

3 初始化与网格拓扑

3.1 基本初始化

```

1 from pyquda import init
2 from pyquda_utils import core
3
4 # 方法1：简单初始化
5 core.init(resource_path=".cache/quda")
6
7 # 方法2：使用pyquda.init
8 from pyquda_comm import initGrid, initDevice
9 from pyquda import initQUDA
10
11 latt_info = core.LatticeInfo([24, 24, 24, 72]) # [Lx, Ly, Lz, Lt
    ]
12 initGrid(latt_info.grid_size)                # 4D进程网格
13 initDevice(0)                                # 选择设备
14 initQUDA()                                   # 初始化QUDA

```

Listing 4: PyQUDA 初始化

3.2 LatticeInfo

```

1 from pyquda_utils.core import LatticeInfo
2
3 # 创建格点信息
4 latt_info = LatticeInfo(
5     [24, 24, 24, 72],          # [Lx, Ly, Lz, Lt]
6     grid_size=[1, 1, 1, 4],   # 4D进程网格
7     t_boundary=-1,            # 时间边界条件（-1：反周期）
8     anisotropy=1.0,           # 各向异性系数

```

```

9  )
10
11 # 访问属性
12 print(latt_info.Gs)    # 空间点总数
13 print(latt_info.Gt)    # 时间点总数
14 print(latt_info.volume) # 总格点数

```

Listing 5: 格点信息对象

3.3 进程网格映射类型

支持以下网格映射策略：

- default: 默认映射
- t_fastest: 时间维度最快变化
- x_fastest: 空间维度最快变化
- minimize: 最小化通信面
- shared: 共享内存映射
- dist_graph: 分布式图映射

4 格点场类型系统

PyQUDA 提供了分层的格点场类型系统。

4.1 场类型层次

```

1  from pyquda_comm.field import (
2      LatticeInfo,          # 格点几何元数据
3      LatticeGauge,         # 规范场 U_mu(x): [Nc, Nc, Nd, Lx, Ly,
                           Lz, Lt]
4      LatticeFermion,       # 费米子场: [Ns, Nc, Lx, Ly, Lz, Lt]
5      LatticePropagator,    # 传播子: [Ns, Nc, Ns, Nc, Lx, Ly, Lz,
                           Lt]
6      LatticeStaggeredFermion, # 交错费米子
7      LatticeMom,           # 动量场 (HMC)
8      LatticeClover,        # Clover 项
9      LatticeRotation,      # 旋转矩阵 (规范固定)

```



```

10     MultiLatticeFermion, # 多源费米子
11 )

```

Listing 6: 格点场类型

4.2 场的基本操作

```

1  # 创建规范场
2  gauge = LatticeGauge(latt_info)
3
4  # 加载规范组态
5  gauge.loadNPYGauge("path/to/config.npy")
6  gauge.loadChromaGauge("path/to/config.lime")
7
8  # 保存规范场
9  gauge.saveNPYGauge("output.npy")
10
11 # 偶-奇预处理
12 gauge_even = gauge.even
13 gauge_odd = gauge.odd
14
15 # 规范化检查
16 checksum = gauge.checksum()

```

Listing 7: 场的创建和操作

4.3 场的类型 (pyquda.field 扩展)

pyquda/field.py 在 pyquda_comm 的基础场上添加了高级方法:

```

1  class LatticeGauge:
2      # HDF5 I/O
3      def loadHDF5Gauge(self, path): ...
4      def saveHDF5Gauge(self, path): ...
5
6      # 规范场操作 (通过gauge_dirac属性)
7      @property
8      def gauge_dirac(self) -> GaugeDirac: ...
9
10     # 涂抹
11     def smearAPE(self, coeff, steps): ...

```

```

12     def smearStout(self, coeff, steps): ...
13     def smearHYP(self, coeff, steps): ...
14
15     # 梯度流
16     def wflow(self, steps, step_size): ...
17     def symanzikFlow(self, steps, step_size): ...
18
19     # 规范固定
20     def gaugeFixingOVR(self, tolerance): ...
21     def gaugeFixingFFT(self): ...
22
23     # 物理观测量
24     def plaquette(self): ...
25     def polyakov(self): ...
26     def topologicalCharge(self): ...

```

Listing 8: LatticeGauge 高级方法

5 Dirac 算子

5.1 算子层次结构

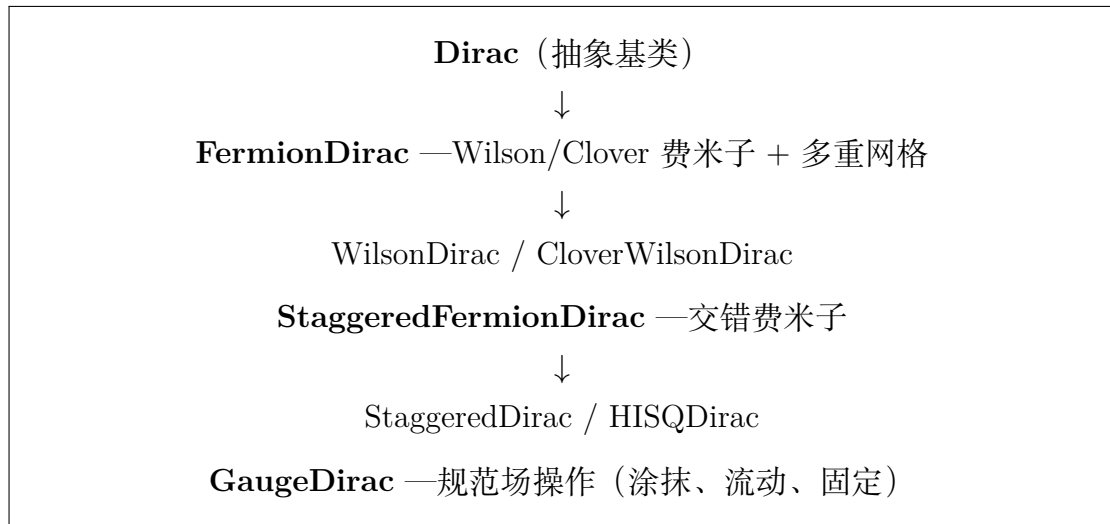


图 2: Dirac 算子继承层次

5.2 创建和使用 Dirac 算子

```

1 from pyquda_utils.core import getWilson, getClover, getHISQ

```

```

2
3 # Wilson费米子
4 wilson_dirac = getWilson(latt_info, gauge, mass=-0.2770)
5
6 # Clover费米子 (自动创建Clover项)
7 clover_dirac = getClover(
8     latt_info, gauge, mass=-0.2770,
9     clover_coeff=1.0,           # clover系数
10    multigrid=True,           # 启用多重网格
11 )
12
13 # HISQ费米子
14 hisq_dirac = getHISQ(latt_info, gauge, mass=0.05)

```

Listing 9: Dirac 算子使用示例

5.3 传播子求解

```

1 from pyquda_utils.core import invert
2 from pyquda_utils.source import wall
3
4 # 墙源传播子
5 prop = invert(
6     clover_dirac,
7     wall([24, 24, 24, 72], latt_info), # 源矢量
8 )
9
10 # 多源传播子
11 props = invert(
12     clover_dirac,
13     [wall([24, 24, 24, 72], latt_info) for _ in range(4)], # 4源
14 )
15
16 # 点源传播子
17 from pyquda_utils.source import point
18 prop = invert(clover_dirac, point(latt_info, [0, 0, 0, 0]))

```

Listing 10: 夸克传播子求解

5.4 Dirac 算子方法

```

1 class FermionDirac:
2     def invert(self, source, mass=None, tol=1e-12, maxiter=10000)
      : ...
3     def invertRestart(self, source, mass=None, ...): ...
4     def invertPC(self, source, mass=None, ...): ...      # 偶-奇
      预处理
5     def invertCloverPC(self, source, mass=None, ...): ...
6     def invertMultiSrc(self, sources, mass=None, ...): ... # 多
      源同时求解
7     def invertMultiSrcRestart(self, sources, mass=None, ...): ...
8     def mat(self, vector): ...      # 矩阵乘法
9     def matDagMat(self, vector): ... # Ddag * D
10    def dslash(self, vector): ...    # Dslash算子
11    def performance(self): ...      # 性能报告

```

Listing 11: FermionDirac 关键方法

5.5 精度控制

全局精度预设:

- none: 不配置精度
- invert: 配置反演精度 (double/half/single 混合)
- multishift: 多 shift 求解精度
- multigrid: 多重网格精度

全局重构预设 (减少内存占用):

- none: 不重构
- wilson: Wilson 费米子 12/8 重构
- staggered: 交错费米子 13/9 重构

6 HMC (混合蒙特卡洛)

6.1 HMC 使用示例

```

1  from math import exp
2  from pyquda.action import GaugeAction
3  from pyquda.action.abstract import LoopParam
4  from pyquda.hmc import HMC, Integrator
5  from pyquda_utils import core
6  from pyquda_utils.core import X, Y, Z, T
7
8  # 初始化
9  core.init(resource_path=".cache/quda")
10 latt_info = core.LatticeInfo([16, 16, 16, 32])
11
12 # 定义规范作用量
13 u_0 = 0.855453; beta = 6.20
14 loop_param = LoopParam(
15     [[X, Y, -X, -Y], [X, Z, -X, -Z], [Y, Z, -Y, -Z],
16      [X, T, -X, -T], [Y, T, -Y, -T], [Z, T, -Z, -T]],
17     [1.0, 1.0, 1.0, 1.0, 1.0, 1.0],
18 )
19
20 # 自定义积分器
21 class O2Nf1Ng0V(Integrator):
22     def integrate(self, updateGauge, updateMom, t):
23         dt = t / self.n_steps
24         for _ in range(self.n_steps):
25             updateMom(dt / 2)
26             updateGauge(dt)
27             updateMom(dt / 2)
28
29 # 创建HMC
30 hmc = HMC(latt_info,
31           [GaugeAction(latt_info, loop_param, beta)],
32           O2Nf1Ng0V(n_steps=100))
33
34 # 运行HMC
35 gauge = core.LatticeGauge(latt_info)
36 hmc.initialize(10086, gauge)
37
38 for i in range(2000):
39     hmc.gaussMom()

```

```

40     energy_old = hmc.momAction() + hmc.gaugeAction()
41     hmc.integrate(1.0, 2e-14)
42     energy = hmc.momAction() + hmc.gaugeAction()
43     accept = hmc.accept(energy - energy_old)
44     if accept or i < 500: # 热化500步
45         hmc.saveGauge(gauge)
46     else:
47         hmc.loadGauge(gauge)

```

Listing 12: 淬火 HMC 示例

6.2 内置积分器

表 2: 内置辛积分器

积分器	说明
INT_3G1F	3 规范力/1 费米子力
O2Nf1Ng0V	2 阶辛, 1 费米子力, 0 规范力, 速度 Verlet
O2Nf1Ng0P	同上, 位置 Verlet
O2Nf2Ng0V	2 阶辛, 2 费米子力
O2Nf2Ng0P	同上, 位置 Verlet
O4Nf5Ng0V	4 阶辛, 5 费米子力
O4Nf5Ng0P	同上, 位置 Verlet

6.3 HMC 类方法

```

1  class HMC:
2      def __init__(self, latt_info, monomials, integrator): ...
3
4      def initialize(self, seed, gauge): ...
5      def gaussMom(self): ...
6      def samplePhi(self): ...
7      def integrate(self, t, tol): ...
8      def accept(self, dE) -> bool: ...
9
10     # 作用量计算
11     def gaugeAction(self) -> float: ...
12     def fermionAction(self) -> float: ...

```

```

13     def momAction(self) -> float: ...
14
15     # 观测量
16     def plaquette(self) -> float: ...
17     def polyakovLoop(self) -> float: ...
18     def energy(self) -> float: ...
19     def qcharge(self) -> float: ...
20     def qchargeDensity(self) -> float: ...
21
22     # 规范场管理
23     def saveGauge(self, gauge): ...
24     def loadGauge(self, gauge): ...

```

Listing 13: HMC 完整接口

6.4 费米子作用量

```

1  # Clover-Wilson 费米子 (1/2 味)
2  from pyquda.action.clover_wilson import CloverWilsonAction
3  clover_action = CloverWilsonAction(latt_info, gauge, mass,
4                                     clover_coeff, n_flavor=2)
5
6  # HISQ 费米子 (N 味)
7  from pyquda.action.hisq import HISQAction
8  hisq_action = HISQAction(latt_info, gauge, mass, n_flavor=4)
9
10 # 多重 HISQ (共享 fat/long 链接)
11 from pyquda.action.hisq import MultiHISQAction
12 multi_hisq = MultiHISQAction(latt_info, gauge, [mass1, mass2],
13                               n_flavor=2)

```

Listing 14: 费米子作用量

7 规范操作

7.1 规范涂抹

通过 GaugeDirac 接口提供 AP、Stout 和 HYP 涂抹。

```

1  # 获取 GaugeDirac

```

```
2  gd = gauge.gauge_dirac
3
4  # APE 涂 抹
5  gauge.smearAPE(coeff=0.5, steps=10)
6  gauge.smearStout(coeff=0.1, steps=10)
7  gauge.smearHYP(coeff=0.5, steps=10)
```

Listing 15: 规范涂抹

7.2 梯度流

```
1  # Wilson 流
2  gauge.wflow(steps=100, step_size=0.01)
3
4  # Symanzik 流
5  gauge.symanzikFlow(steps=100, step_size=0.01)
6
7  # 流标度确定 (t0, w0)
8  from pyquda.dirac.gauge import GaugeDirac
9  gd = GaugeDirac(latt_info)
10 gd.wflow_scale_determination(gauge, steps=100, step_size=0.01)
```

Listing 16: 梯度流

7.3 规范固定

```
1  # 过松弛方法 (OVR)
2  gauge.gaugeFixingOVR(tolerance=1e-12)
3
4  # 快速傅里叶变换方法 (FFT)
5  gauge.gaugeFixingFFT()
```

Listing 17: 规范固定

7.4 物理观测量

```
1  # Plaquette
2  plaq = gauge.plaquette()
3
4  # Polyakov 圈
```



```

5  poly = gauge.polyakov()
6
7  # 拓扑荷
8  Q = gauge.topologicalCharge()
9
10 # 规范作用量密度
11 E = gauge.energy()

```

Listing 18: 规范观测量

8 文件 I/O

PyQUDA 支持 10 余种格点数据格式的读写。

8.1 支持的格式

表 3: 文件 I/O 格式支持矩阵

格式	规范读	规范写	传播子读	传播子写
Chroma (SciDAC/LIME)	✓	—	✓	—
MILC	✓	✓	✓	—
KYU	✓	✓	✓	✓
XQCD	—	—	✓	✓
NERSC	✓	✓	—	—
OpenQCD	✓	✓	—	—
ILDG	✓	—	—	—
NPY (numpy)	✓	✓	✓	✓
HDF5 (h5py)	✓	✓	✓	✓
General (自定义)	✓	✓	—	—

8.2 加载/保存规范组态

```

1  from pyquda_io import chroma, milc, npy
2
3  # Chroma 格式
4  gauge = chroma.readChromaGauge("config.lime", latt_info)
5
6  # MILC 格式

```

```

7 gauge = milc.readMILCGauge("config.milc", latt_info)
8
9 # KYU 格式
10 from pyquda_io import kyu
11 kyu.writeKYUGauge("config.kyu", gauge)
12
13 # NPY 格式
14 npy.writeNPYGauge("config.npy", gauge)
15 gauge = npy.readNPYGauge("config.npy", latt_info)
16
17 # HDF5 格式
18 gauge.saveHDF5Gauge("config.h5")
19 gauge.loadHDF5Gauge("config.h5")

```

Listing 19: 规范组态 I/O

8.3 校验和

```

1 from pyquda_utils.checksum import checksum
2
3 # SciDAC 风格 校验和 (CRC32 + rank-based XOR)
4 sum_a, sum_b = gauge.checksum()

```

Listing 20: 校验和验证

9 Gamma 矩阵系统

```

1 from pyquda.gamma import Gamma, Projector
2
3 # 基本 Gamma 矩阵
4 g5 = Gamma(15)      # gamma_5
5 g0 = Gamma(0)       # gamma_t (DeGrand-Rossi 基)
6 g1 = Gamma(1)       # gamma_x
7 g2 = Gamma(2)       # gamma_y
8 g3 = Gamma(3)       # gamma_z
9
10 # Gamma 矩阵运算
11 g_plus = (g0 + g3) / 2  # (1+gamma_3)/2 投影算符
12 g5_g0 = g5 @ g0        # gamma_5 * gamma_t

```

```

13 g5g0_T = g5_g0.T           # 转置
14 g5g0_H = g5_g0.H           # 厄米共轭
15
16 # 自定义投影算符
17 proj = Projector([
18     0.5, 0, 0, 0.5,         # 第一行
19     0, 0, 0, 0,
20     0, 0, 0, 0,
21     0.5, 0, 0, 0.5,         # 最后一行
22 ]) # (1+gamma_3)/2

```

Listing 21: Γ 矩阵运算

9.1 $\Gamma \times$ 传播子

pyquda_utils/gamma.py 通过 monkey-patching 为 LatticeFermion 和 LatticePropagator 添加矩阵乘法:

```

1 from pyquda_utils.gamma import * # 自动添加 __matmul__ 到场类型
2
3 # gamma_5 @ 传播子
4 result = Gamma(15) @ prop
5
6 # 投影算符 @ 费米子场
7 projected = Projector(...) @ fermion

```

Listing 22: $\Gamma \times$ 传播子操作

10 源构造

pyquda_utils/source.py 提供了多种夸克源构造方法:

```

1 from pyquda_utils.source import (
2     point, wall, volume, momentum, colorvector, source, source12,
3     source3
4 )
5
6 # 点源
7 src = point(latt_info, position=[0, 0, 0, 0])
8
9 # 墙源 (时间片0上的所有空间点)

```

```

9  src = wall([24, 24, 24, 72], latt_info, t=0)
10
11  # 动量源 (带Fourier相位)
12  src = momentum(latt_info, mom=[0, 0, 1, 0])
13
14  # 体积源
15  src = volume([24, 24, 24, 72], latt_info)
16
17  # 颜色矢量源
18  src = colorvector(latt_info, color=0, spin=0)
19
20  # 一般源 (12分量: 自旋*颜色)
21  src = source12(latt_info) # 无涂抹
22  src = source12(latt_info, source_type="gaussian") # 高斯涂抹

```

Listing 23: 源构造方法

10.1 高斯/Wuppertal 涂抹

```

1  # 高斯涂抹源
2  from pyquda.dirac.gauge import GaugeDirac
3  gd = GaugeDirac(latt_info)
4  gd.gaussianSmearSource(source, gauge, n_steps=50, sigma=4.0)

```

Listing 24: 高斯涂抹

11 插件系统

PyQUDA 提供了一个自动化的 C/CUDA 插件系统, 从 C 头文件自动生成 Cython 封装。

11.1 内置插件

- **pycontract**: GPU 加速的介子/重子收缩函数
- **pygwu**: Grid-Wilson-Overlap 接口

11.2 pycontract —夸克收缩

```

1 from pyquda_plugins.pycontract import (
2     mesonTwoPoint, mesonAllSinkTwoPoint,
3     baryonTwoPoint, baryonDiquark,
4     baryonSequentialTwoPoint,
5 )
6
7 # 介子两点函数
8 C2pt = mesonTwoPoint(prop, prop, gamma_list)
9
10 # 重子两点函数
11 C2pt_baryon = baryonTwoPoint(prop, prop, gamma_list)

```

Listing 25: pycontract 收缩函数

11.3 pygwu —Overlap 费米子

```

1 from pyquda_plugins.pygwu import Overlap
2
3 ov = Overlap(latt_info, gauge, mass=-0.2770, kappa=0.15)
4 ov.buildWilsonKernel()
5 ov.invert(source, mass=0.01, tol=1e-12)

```

Listing 26: pygwu Overlap 接口

12 示例演示

12.1 示例 1：淬火 HMC

examples/1_Quenched_HMC.py —纯规范场 HMC 模拟, 在 $16^3 \times 32$ 格点上, $\beta = 6.20$, 使用 Wilson 规范作用量和 O2Nf1Ng0V 积分器。

```

1 cd PyQUDA/examples
2 python 1_Quenched_HMC.py
3 # 输出: 每5条轨迹保存NPY格式规范组态到 ./cfg/

```

Listing 27: 运行淬火 HMC

12.2 示例 2: 夸克传播子与 介子两点函数

`examples/2_Quark_Propagator.py` — 读取 Chroma 规范组态, 施加 Stout 涂抹, 使用多重网格 Clover-Wilson Dirac 求解器, 从墙源传播子计算 介子两点函数。

12.3 示例 3: 介子和质子两点函数

`examples/3_Pion_Proton_2pt.py` — 在示例 2 的基础上增加质子两点函数的计算, 使用标准的颜色收缩重子算符和宇称投影。

12.4 示例 4: PCAC 验证

`examples/4_Pion_PCAC.py` — 通过计算 介子轴矢量关联函数验证 PCAC 关系。

12.5 示例 5: 介子色散关系

`examples/5_Pion_Dispersion.py` — 使用动量投影计算不同动量下的 介子能量, 验证连续色散关系 $E^2 = m_\pi^2 + p^2$ 。

13 QUDA 底层 API

13.1 参数结构体

PyQUDA 暴露了 QUDA 的所有主要参数结构体:

```

1  # 规范场参数
2  from pyquda import QudaGaugeParam
3  gauge_param = QudaGaugeParam()
4  gauge_param.cpu_prec = 2          # 双精度
5  gauge_param.cuda_prec = 2        # 双精度
6  gauge_param.reconstruct = 18     # 无重构
7  gauge_param.t_boundary = -1      # 反周期时间边界
8
9  # 反演参数
10 from pyquda import QudaInvertParam
11 invert_param = QudaInvertParam()
12 invert_param.tol = 1e-12          # 收敛精度
13 invert_param.maxiter = 10000      # 最大迭代次数
14 invert_param.solve_type = 2       # 直接求解 (Direct)
15 invert_param.solution_type = 1    # 无预处理 (Mat)
16 invert_param.dslash_type = 1     # Wilson dslash

```

```

17 invert_param.residual_type = 2      # L2相对残差
18
19 # 本征求解器参数
20 from pyquda import QudaEigParam
21
22 # 多重网格参数
23 from pyquda import QudaMultigridParam

```

Listing 28: QUDA 参数结构体

13.2 主要 QUDA 函数

```

1 from pyquda import (
2     # 初始化
3     initQuda, endQuda, initCommsGridQuda,
4
5     # 规范场
6     loadGaugeQuda, saveGaugeQuda, freeGaugeQuda,
7     loadCloverQuda, freeCloverQuda,
8     gaussGaugeQuda, gaussMomQuda,
9     projectSU3Quda,
10    computeGaugeForceQuda,
11    updateGaugeFieldQuda,
12
13    # 观测量
14    plaqQuda, polyakovLoopQuda, gaugeObservablesQuda,
15
16    # 求解器
17    invertQuda, invertMultiSrcQuda, invertMultiShiftQuda,
18    dslashQuda, dslashMultiSrcQuda,
19    MatQuda, MatDagMatQuda,
20    cloverQuda,
21
22    # 多重网格 / 本征求解
23    newMultigridQuda, destroyMultigridQuda, updateMultigridQuda,
24    eigensolveQuda, newDeflationQuda, destroyDeflationQuda,
25
26    # 涂抹 / 流
27    performGaugeSmearQuda, performWFlowQuda,
28    performWuppertalnStep, performTwoLinkGaussianSmearNStep,

```

```

29
30     # 力计算
31     computeCloverForceQuda, computeHISQForceQuda,
32
33     # 规范固定
34     computeGaugeFixingOVRQuda, computeGaugeFixingFFTQuda,
35
36     # 其他
37     momActionQuda, computeKSLinkQuda,
38     contractQuda, blasGEMMQuda,
39     flushChronoQuda, setVerbosityQuda,
40 )

```

Listing 29: 直接调用的 QUDA 函数

14 高级功能

14.1 动量相位

```

1  from pyquda_utils.phase import MomentumPhase, GridPhase
2
3  # 动量投影相位
4  phase = MomentumPhase([24, 24, 24, 72], latt_info, mom=[0, 0, 1,
5  0])
6
7  prop_p = phase @ prop # 动量投影传播子
8
9  # 格点步幅相位
10 grid_phase = GridPhase(latt_info, step=[2, 2, 2, 1])

```

Listing 30: 动量相位

14.2 Wilson 圈计算

```

1  from pyquda_utils.wilson_loop import wilson_loop
2
3  W = wilson_loop(gauge, R=3, T=4, dirs=["X", "Y", "Z"])

```

Listing 31: Wilson 圈

14.3 规范场转换

```
1 from pyquda_utils.convert import gauge_convert
2
3 # SU(3) 规范场转换为SU(2)子群
4 gauge_su2 = gauge_convert(gauge, "su3_to_su2")
```

Listing 32: 规范场格式转换

14.4 FFT 工具

```
1 from pyquda_utils.fft import fft, ifft
2
3 # 格点场的快速傅里叶变换
4 freq = fft(fermion_field)
```

Listing 33: FFT 操作

15 API 接口清单

15.1 初始化模块 (pyquda/___init___py)

```
1 def init(
2     resource_path: str,
3     grid_type: str = "default",
4     device: int = 0,
5     enable_quda: bool = True,
6     **grid_kwargs,
7 ) -> None: ...
8
9 def initGrid(grid_size: list[int], grid_type: str = "default"):
10     ...
11
12 def initDevice(device: int): ...
13
14 def initQUDA(): ...
15
16 def isQUDAInitialized() -> bool: ...
17
18 def end(): ...
```

Listing 34: 初始化函数

15.2 场模块 (pyquda_comm/field.py)

```

1 class LatticeInfo:
2     def __init__(self, dims, grid_size=None, t_boundary=-1,
3         anisotropy=1.0): ...
4     # 属性: Gs, Gt, volume, grid_size, dims
5
6 class LatticeGauge(BaseField):
7     def loadNPYGauge(self, path): ...
8     def saveNPYGauge(self, path): ...
9     def loadHDF5Gauge(self, path): ...
10    def saveHDF5Gauge(self, path): ...
11    def smearAPE(self, coeff, steps): ...
12    def smearStout(self, coeff, steps): ...
13    def smearHYP(self, coeff, steps): ...
14    def wflow(self, steps, step_size): ...
15    def symanzikFlow(self, steps, step_size): ...
16    def gaugeFixingOVR(self, tolerance): ...
17    def gaugeFixingFFT(self): ...
18    def plaquette(self) -> float: ...
19    def polyakov(self) -> float: ...
20    def topologicalCharge(self) -> float: ...
21    def energy(self) -> float: ...
22    @property
23    def gauge_dirac(self) -> GaugeDirac: ...
24
25 class LatticeFermion(BaseField): ...
26 class LatticePropagator(BaseField): ...
27 class LatticeMom(BaseField): ...
28 class LatticeClover(BaseField): ...
29 class LatticeRotation(BaseField): ...

```

Listing 35: 场基类

15.3 Dirac 算子模块 (pyquda/dirac/)

```

1 class Dirac:
2     def __init__(self, latt_info, gauge_param=None, invert_param=
3         None): ...

```

```

4 class FermionDirac(Dirac):
5     def invert(self, source, mass=None, tol=1e-12, maxiter=10000)
      : ...
6     def invertRestart(self, source, mass=None): ...
7     def invertPC(self, source, mass=None): ...
8     def invertCloverPC(self, source, mass=None): ...
9     def invertMultiSrc(self, sources, mass=None): ...
10    def invertMultiSrcRestart(self, sources, mass=None): ...
11    def mat(self, vector): ...
12    def matDagMat(self, vector): ...
13    def dslash(self, vector): ...
14    def performance(self): ...
15
16 class WilsonDirac(FermionDirac): ...
17 class CloverWilsonDirac(FermionDirac): ...
18 class StaggeredDirac(StaggeredFermionDirac): ...
19 class HISQDirac(StaggeredFermionDirac): ...
20
21 class GaugeDirac(Dirac):
22     def covDev(self, field, dir, displacement): ...
23     def laplacian(self, field): ...
24     def smearWuppertal(self, source, n_steps, sigma): ...
25     def smearGaussian(self, source, n_steps, sigma): ...
26     def projectSU3(self, gauge): ...
27     def gaugePath(self, path): ...
28     def gaugeLoopTrace(self, path): ...
29     def wflow_scale_determination(self, gauge, steps, step_size):
      ...

```

Listing 36: Dirac 算子接口

15.4 作用量模块 (pyquda/action/)

```

1 class Action:
2     def action(self) -> float: ...
3     def force(self) -> float: ...
4
5 class GaugeAction(Action):
6     def __init__(self, latt_info, loop_param: LoopParam, beta:
      float): ...

```

```

7
8 class FermionAction(Action):
9     def __init__(self, latt_info, gauge, mass, n_flavor, ...):
10         ...
11     def sample(self): ...
12     def invertMultiShift(self, source, masses): ...
13
14 class CloverWilsonAction(FermionAction):
15     def __init__(self, latt_info, gauge, mass, clover_coeff,
16         n_flavor): ...
17
18 class HISQAction(StaggeredFermionAction):
19     def __init__(self, latt_info, gauge, mass, n_flavor): ...
20
21 class LoopParam:
22     def __init__(self, paths: list[list[int]], coeffs: list[float
23         ]): ...

```

Listing 37: 作用量接口

15.5 HMC 模块 (pyquda/hmc.py)

```

1 class HMC:
2     def __init__(self, latt_info, monomials: list[Action],
3         integrator: Integrator): ...
4     def initialize(self, seed: int, gauge: LatticeGauge): ...
5     def gaussMom(self): ...
6     def samplePhi(self): ...
7     def integrate(self, t: float, tol: float): ...
8     def accept(self, dE: float) -> bool: ...
9     def gaugeAction(self) -> float: ...
10    def fermionAction(self) -> float: ...
11    def momAction(self) -> float: ...
12    def plaquette(self) -> float: ...
13    def polyakovLoop(self) -> float: ...
14    def energy(self) -> float: ...
15    def qcharge(self) -> float: ...
16    def qchargeDensity(self) -> float: ...
17    def saveGauge(self, gauge: LatticeGauge): ...
18    def loadGauge(self, gauge: LatticeGauge): ...

```

```

18
19 class Integrator:
20     def __init__(self, n_steps: int): ...
21     def integrate(self, updateGauge, updateMom, t: float): ...

```

Listing 38: HMC 接口

15.6 Gamma 矩阵模块 (pyquda/gamma.py)

```

1 class Gamma:
2     def __init__(self, index: int, factor=1.0): ...
3     # index: 0-15 (DeGrand-Rossi 基)
4     # 支持: +, -, *, @, .T (转置), .H (厄米共轭), .D (dagger)
5     @property
6     def matrix(self): ...          # 4x4 numpy 数组
7     @property
8     def sparse_indices(self): ...
9     @property
10    def sparse_data(self): ...
11
12 class Projector:
13     def __init__(self, factors: list[float]): ... # 长度16
14     # 支持: +, -, *, @, .T, .H

```

Listing 39: Gamma 矩阵接口

15.7 源构造模块 (pyquda_utils/source.py)

```

1 def point(latt_info, position): ...          # 点源
2 def wall(dims, latt_info, t=0): ...          # 墙源
3 def volume(dims, latt_info): ...             # 体积源
4 def momentum(latt_info, mom): ...            # 动量源
5 def colorvector(latt_info, color, spin): ... # 颜色矢量源
6 def source(latt_info): ...                   # 一般源
7 def source12(latt_info, source_type=None): ... # 12分量源 (自旋+
    颜色)
8 def source3(latt_info, source_type=None): ... # 3分量源 (交错费
    米子)

```

Listing 40: 源构造接口

15.8 I/O 模块 (pyquda_io/)

```

1  # chroma.py
2  def readChromaGauge(path: str, latt_info) -> LatticeGauge: ...
3  def readChromaPropagator(path: str, latt_info) ->
    LatticePropagator: ...
4
5  # milc.py
6  def readMILCGauge(path: str, latt_info) -> LatticeGauge: ...
7  def writeMILCGauge(path: str, gauge): ...
8  def readMILCPropagator(path: str, latt_info) -> LatticePropagator
    : ...
9
10 # npy.py
11 def readNPYGauge(path: str, latt_info) -> LatticeGauge: ...
12 def writeNPYGauge(path: str, gauge): ...
13
14 # kyu.py, xqcd.py, nersc.py, openqcd.py, ildg.py
15 # 类似的 read/write 接口
16
17 # general.py
18 def readGeneralGauge(path: str, latt_info) -> LatticeGauge: ...
19 def writeGeneralGauge(path: str, gauge): ...

```

Listing 41: I/O 接口

15.9 工具模块 (pyquda_utils/core.py)

```

1  def init(resource_path=".cache/quda", grid_type="default", device
    =0): ...
2  def invert(dirac, source, mass=None, tol=1e-12, maxiter=10000):
    ...
3  def getDirac(latt_info, gauge, clover_coeff=None, mass=-0.2770):
    ...
4  def getWilson(latt_info, gauge, mass=-0.2770): ...
5  def getClover(latt_info, gauge, mass=-0.2770, clover_coeff=1.0,
    multigrid=True): ...
6  def getHISQ(latt_info, gauge, mass=0.05): ...
7  def getStaggeredDirac(latt_info, gauge, mass=0.05): ...
8  def gatherLattice(field) -> np.ndarray: ... # MPI 收集+归约

```

Listing 42: core.py 便捷函数

15.10 插件模块 (pyquda_plugins/)

```

1  # pycontract
2  def mesonTwoPoint(prop1, prop2, gamma_list): ...
3  def mesonAllSinkTwoPoint(prop1, prop2, gamma_list): ...
4  def baryonTwoPoint(prop1, prop2, gamma_list): ...
5  def baryonDiquark(prop1, prop2, gamma_list): ...
6  def baryonSequentialTwoPoint(prop1, prop2, seq_prop, gamma_list):
    ...
7
8  # pygwu
9  class Overlap:
10     def __init__(self, latt_info, gauge, mass, kappa): ...
11     def buildWilsonKernel(self): ...
12     def invert(self, source, mass, tol=1e-12): ...

```

Listing 43: 插件接口

16 性能基准测试

PyQUDA 的基准测试说明参见项目 Wiki: <https://github.com/CLQCD/PyQUDA/wiki/Benchmark>

QUDA 环境变量 (30+) 可用于精细调节 GPU 性能:

- QUDA_RESOURCE_PATH: QUDA 资源缓存路径
- QUDA_ENABLE_P2P: 启用 GPU 对等通信
- QUDA_ENABLE_GDR: 启用 GPU 直连 RDMA
- QUDA_MPI_REDUCE_LOCAL_COPY: MPI 归约的本地拷贝优化

17 维护说明

- 函数定义 (主要在 quda.in.pxd 和 quda.in.pyx 中) 及大部分文档字符串 (主要在 pyquda.pyi 和 enum_quda.in.py 中) 需要手动更新, 因为目前无法自动生成

- PyQUDA 应与 QUDA 未来的 API 兼容版本配合良好，前提是 QUDA 的 API 保持不变
- 核心 Cython 绑定 (`quda.pyx`) 从 QUDA 的 C 头文件使用 `pycparser` 自动生成，插件系统也采用同样的机制

17.1 更新 QUDA API

当 QUDA 更新时，需要手动同步以下文件：

- `pyquda/quda.pyi`: Python 类型存根
- `pyquda/quda.in.pxd`: Cython 声明
- `pyquda/quda.in.pyx`: Cython 实现
- `pyquda/enum_quda.in.py`: 枚举类型